SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA5112

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Scenario-Based : 20%

QUESTIONS:5 **Total Marks:100**

Annexure A

Code of Conduct:

I P. Nikitha -192324226*_certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student :

Annexure A
Scenario-based

1. Secure Data Transmission using Block Cipher
A company is transmitting repeated financial data blocks over a network. Initially, they used ECB mode and noticed patterns in ciphertext.

A. Explain why this issue occurs.
B. Suggest a more secure block cipher mode and justify your choice based on the scenario.

2. Choosing an Encryption Algorithm for Banking System
A banking application needs high security and moderate performance for encrypting customer transactions. The available options are DES, 3-DES, and Blowfish.

A. Identify the most suitable algorithm for this scenario.
B. Justify your choice based on security strength and efficiency.

3. Key Exchange in an Unsecured Network
Two users want to establish a secure communication channel over a public network using Diffie-Hellman key exchange. However, the network is vulnerable to attacks.

A. Explain how the key exchange works in this scenario.
B. Identify a possible security threat and suggest a solution to mitigate it.

4. Public Key Cryptography for Email Security
An organization plans to implement RSA for secure email communication.

A. Describe how RSA can be used for encryption and decryption in this scenario.
B. Explain how key distribution and management play a role in maintaining security.

5. Cryptography for IoT Devices
A smart home system uses low-power IoT devices that need secure communication. The designer must choose between RSA and Elliptic Curve Cryptography (ECC).

A. Analyze the constraints of IoT devices.
B. Recommend a suitable cryptographic approach and justify your answer.

1. Secure Data Transmission Using Block Cipher

Scenario: A company transmits repeated financial data blocks over a network. Initially, it uses Electronic Codebook (ECB) mode, but patterns appear in the ciphertext.

1.1 Explain why this issue occurs.

The issue occurs because Electronic Codebook (ECB) mode encrypts each plaintext block independently using the same secret key. If two plaintext blocks contain identical data, their ciphertext blocks will also be identical.

Block ciphers, such as AES, process data in fixed-size blocks. For example, AES uses a block size of 128 bits. When a large message is encrypted, it is divided into multiple blocks before encryption.

In ECB mode, the encryption process is represented as:

$$C_i = E_K(P_i)$$

Where:

- P_i = the i-th plaintext block.
- C_i = the corresponding ciphertext block.
- E_K = encryption using secret key K.

If two plaintext blocks are identical, then:

$$P_1 = P_2 \Rightarrow C_1 = C_2$$

This relationship allows an attacker to identify repeated patterns in the encrypted data, even without knowing the encryption key.

Example:

Suppose a company transmits the following financial records:

Plaintext block	**ECB ciphertext pattern**
Transaction amount: ₹5,000	Ciphertext block A
Transaction amount: ₹8,000	Ciphertext block B
Transaction amount: ₹5,000	Ciphertext block A
Transaction amount: ₹5,000	Ciphertext block A

Because the first, third, and fourth blocks contain identical plaintext, they produce identical ciphertext blocks under the same key.

An attacker monitoring the network may recognize repeated blocks and infer information about the structure or repetition of the financial records.

ECB does not necessarily reveal the actual plaintext values directly, but it leaks patterns and relationships between blocks.

Main disadvantages of ECB mode:

1. Pattern leakage: Identical plaintext blocks produce identical ciphertext blocks.
2. Lack of semantic confidentiality: An attacker can infer repeated data and message structure.
3. Block manipulation: An attacker may rearrange or substitute ciphertext blocks. ECB does not provide integrity or authentication.
4. Unsuitability for structured data: Repeated transaction fields, account records, and other predictable information can reveal patterns.

Therefore, ECB is generally unsuitable for encrypting sensitive financial information transmitted over a network.

1.2 Suggest a more secure block cipher mode and justify your choice.

The recommended choice is AES in Galois/Counter Mode (AES-GCM).

AES-GCM combines encryption with authentication. It protects the confidentiality of financial data and helps detect unauthorized modifications.

If the question is specifically comparing traditional block cipher modes, Cipher Block Chaining (CBC) is also an improvement over ECB for confidentiality when implemented correctly. However, AES-GCM is the stronger practical recommendation because it provides both encryption and integrity protection.

How AES-GCM works

AES-GCM uses a unique nonce (number used once) for each encryption under the same key.

Its encryption process can be represented as:

$$(C,T)=\text{AES-GCM}_K(N,P,A)$$

Where:

- K = secret encryption key.
- N = unique nonce.
- P = plaintext financial data.
- A = optional additional authenticated data, such as transaction metadata.
- C = ciphertext.
- T = authentication tag.

Even when the same plaintext is encrypted repeatedly using the same key, different nonces produce different ciphertexts, assuming correct implementation.

The receiver uses the secret key, nonce, ciphertext, and authentication tag to verify and decrypt the message. If the ciphertext or authenticated data has been modified, authentication

fails and the message must be rejected.

Advantages of AES-GCM

- Confidentiality: Protects financial records from unauthorized access.
- Pattern protection: Prevents ECB-style repetition leakage when nonces are correctly managed.
- Integrity: Detects unauthorized changes to ciphertext.
- Authentication: Verifies that the encrypted data has not been tampered with.
- Performance: Supports efficient implementations in modern processors.

Important implementation requirement: A nonce must never be reused with the same AES-GCM key. Reuse can seriously compromise security. The system must also verify the authentication tag before accepting decrypted data.

Conclusion: AES-GCM is the most suitable choice because the company needs to protect repeated financial data against pattern leakage, unauthorized disclosure, and tampering during network transmission.

2. Choosing an Encryption Algorithm for a Banking System

Scenario: A banking application requires high security and moderate performance. The available algorithms are DES, Triple DES (3DES), and Blowfish.

2.1 Identify the most suitable algorithm.

Among the three listed options, Blowfish is generally the best choice for a new implementation, primarily because DES and 3DES have significant security or performance limitations.

However, an important practical qualification is that none of these three should be the default choice for a new banking system today. Modern banking applications should generally use AES, preferably AES-GCM for authenticated encryption, according to applicable security requirements.

If the examination requires selecting only from DES, 3DES, and Blowfish, Blowfish is the most defensible answer for the stated comparison, subject to the limitations discussed below.

2.2 Justify your choice based on security strength and efficiency.

The three algorithms can be compared as follows.

Feature	DES	Triple DES (3DES)	Blowfish
Block size	64 bits	64 bits	64 bits
Key strength	56 effective key bits	Historically stronger than DES	Variable key length, up to 448 bits

Security status	Insecure against brute-force attacks	Legacy; deprecated for new encryption	Older design with no practical break of the full cipher widely established, but not ideal for new systems
Performance	Fast, but insecure	Slower because it performs DES operations multiple times	Efficient in software in many implementations
Main limitation	Very small key	Small block size and legacy status	Small block size and lack of modern AEAD support

A. DES (Data Encryption Standard)

DES is a symmetric block cipher that uses a 56-bit effective key and a 64-bit block size.

Its principal weakness is its short key length.

An attacker can attempt all possible keys through a brute-force search. With only 2^{56} possible keys, DES no longer provides adequate security against modern computing capabilities.

Although DES can be efficient in suitable hardware, its security weakness makes it inappropriate for protecting current banking transactions.

Conclusion: DES should not be selected because it does not provide sufficient protection for sensitive financial information.

B. Triple DES (3DES)

3DES applies the DES algorithm three times to each data block, commonly using an encrypt-decrypt-encrypt sequence.

Its simplified structure is:

$$C = E_{K_3}(D_{K_2}(E_{K_1}(P)))$$

Where:

- P = plaintext.
- C = ciphertext.
- K_1, K_2, K_3 = DES keys.
- E = encryption.
- D = decryption.

By applying DES multiple times, 3DES improves resistance to attacks that defeat ordinary DES. Its effective security depends on the keying option and attack model; it should not simply be described as providing 168 bits of effective security.

However, 3DES has two major disadvantages:

1. It is substantially slower than a single-pass block cipher because it performs multiple DES operations.
2. Its 64-bit block size creates collision risks when large volumes of data are encrypted under the same key.

3DES has been deprecated for new encryption applications by major standards bodies, so it is unsuitable as the preferred algorithm for a modern banking system.

Conclusion: 3DES offers historical improvements over DES but is not a good modern choice because of its performance and block-size limitations.

C. Blowfish

Blowfish is a symmetric block cipher designed by Bruce Schneier. It supports variable key lengths from 32 to 448 bits and uses a 64-bit block size.

Its advantages include:

- Flexible key lengths.
- Efficient encryption and decryption in many software implementations.
- A well-studied design with no widely established practical attack that breaks the full cipher under ordinary usage assumptions.

However, Blowfish is an older algorithm. Its 64-bit block size is a significant limitation when encrypting large amounts of data, because repeated blocks and birthday-bound collision risks become increasingly relevant as data volume grows.

It also does not natively provide the authenticated-encryption functionality offered by modern modes such as AES-GCM.

Conclusion: Among the three choices, Blowfish is a reasonable answer in a historical or academic comparison because it avoids DES's short-key weakness and is often efficient in software. Nevertheless, it should not be the preferred cipher for a new banking application.

Final recommendation

For an examination answer, select Blowfish among the three listed algorithms, explaining its stronger key-length options and software efficiency compared with DES and 3DES.

For an actual banking system, select AES-256-GCM, or another approved AES configuration appropriate to the system's security requirements.

AES provides a modern block size of 128 bits, strong security, efficient implementations, and authenticated encryption when used with GCM. The key must be protected appropriately, nonces must be managed correctly, and the implementation should follow the applicable banking and regulatory standards.

3. Key Exchange in an Unsecured Network

Scenario: Two users want to establish a secure communication channel over a public network using Diffie–Hellman key exchange, but the network is vulnerable to attacks.

3.1 Explain how the key exchange works in this scenario.

The Diffie–Hellman (DH) key exchange protocol allows two parties to establish a shared secret over a public network without transmitting that secret directly.

The protocol uses mathematical operations involving modular exponentiation. Its security relies on the difficulty of solving the discrete logarithm problem for appropriately selected parameters.

Assume that Alice and Bob want to communicate securely.

Step 1: Agree on public parameters

Alice and Bob publicly agree on two values:

- A large prime number p.
- A generator g of a suitable subgroup modulo p.

These values are public and do not need to be kept secret.

Step 2: Alice selects a private value

Alice chooses a random private integer a.

She calculates her public value:

$$A = g^a \bmod p$$

Alice sends A to Bob over the public network.

Step 3: Bob selects a private value

Bob chooses a random private integer b.

He calculates his public value:

$$B = g^b \bmod p$$

Bob sends B to Alice.

Step 4: Alice calculates the shared secret

Alice receives Bob's public value B and calculates:

$$S_A = B^a \bmod p$$

Substituting $B = g^b \bmod p$:

$$S_A = (g^b)^a \bmod p$$

Therefore:

$$S_A = g^{ab} \bmod p$$

Step 5: Bob calculates the shared secret

Bob receives Alice's public value A and calculates:

$$S_B = A^b \bmod p$$

Substituting $A = g^a \bmod p$:

$$S_B = (g^a)^b \bmod p$$

Therefore:

$$S_B = g^{ab} \bmod p$$

Both parties obtain the same shared secret:

$$S_A = S_B = g^{ab} \bmod p$$

The shared secret is normally passed through a key derivation function (KDF) to produce one or more cryptographic keys. Those keys can then be used by an authenticated encryption algorithm, such as AES-GCM, to protect the actual messages.

Important: The raw Diffie–Hellman secret should not normally be used directly as an encryption key.

3.2 Identify a possible security threat and suggest a solution.

The most important threat in this scenario is a Man-in-the-Middle (MITM) attack.

How the MITM attack works

Although Diffie–Hellman can establish a shared secret, ordinary unauthenticated Diffie–Hellman does not verify the identity of the other party.

An attacker, Mallory, can intercept the exchange:

1. Alice sends her public value *A*, but Mallory intercepts it.
2. Mallory sends her own public value to Bob instead of Alice's value.
3. Bob sends his public value *B*, but Mallory intercepts it.
4. Mallory sends another attacker-controlled public value to Alice.
5. Alice establishes a shared secret with Mallory.
6. Bob independently establishes a different shared secret with Mallory.

Mallory now has one shared secret with Alice and another with Bob. She can decrypt messages from one side, read or modify them, and re-encrypt them for the other side.

Neither Alice nor Bob may realize that the communication has been intercepted.

Solution: Authenticate the key exchange

The best mitigation is to combine Diffie–Hellman with an authentication mechanism.

Possible solutions include:

- Digital certificates: Use certificates issued by a trusted Certificate Authority (CA) to verify public-key identities.
- Digital signatures: Sign the Diffie–Hellman exchange using a trusted private signing key, then verify the signature using the corresponding authenticated public key.
- Authenticated protocols: Use TLS 1.3 with certificate-based authentication, or another appropriately configured authenticated key-exchange protocol.
- Public-key fingerprint verification: For suitable applications, independently compare

trusted fingerprints through a separate communication channel.

The protocol should also validate public-key parameters and public values, use strong ephemeral keys where appropriate, and derive session keys securely.

Conclusion: Diffie–Hellman solves the problem of establishing a shared secret over a public network, but it does not independently guarantee the identities of the participants. Authentication is necessary to prevent a MITM attack.

4. Public Key Cryptography for Email Security

Scenario: An organization plans to use RSA to secure email communications.

4.1 Describe how RSA can be used for encryption and decryption.

RSA is an asymmetric cryptographic algorithm. Unlike symmetric encryption, which uses the same secret key for encryption and decryption, RSA uses a mathematically related public key and private key.

- Public key: Can be shared with other users.
- Private key: Must remain confidential and under the owner's control.

For example, Alice wants to send a confidential email to Bob.

Step 1: Bob generates an RSA key pair

Bob generates an RSA public/private key pair using a secure implementation.

The public key contains the RSA modulus n and public exponent e. The private key contains the corresponding secret exponent d, together with other private parameters.

The key pair is mathematically related so that a correctly formed RSA encryption operation can be reversed using the corresponding private key.

Step 2: Bob shares his public key

Bob distributes his public key to Alice, for example through a trusted directory, certificate, or secure organizational key-management system.

Alice must verify that the public key actually belongs to Bob. Simply downloading a public key from an unverified source is not sufficient.

Step 3: Alice encrypts the email

In a simplified mathematical description, RSA encryption is:

$$C = M^e \bmod n$$

Where:

- M = encoded message representative.
- C = RSA ciphertext.
- e = public exponent.
- n = RSA modulus.

In real systems, RSA encryption must use a secure padding scheme, such as RSA-OAEP, rather than applying the raw mathematical operation directly to an arbitrary message.

Practical consideration: RSA is not normally used to encrypt an entire email directly. Emails may be too large, and public-key operations are comparatively expensive.

Instead, a hybrid encryption approach is used.

1. Alice generates a random symmetric session key.
2. Alice encrypts the email body and attachments using an authenticated symmetric algorithm, such as AES-GCM.
3. Alice protects the session key for Bob using Bob's public key, or uses an established secure email protocol that provides an equivalent key-wrapping mechanism.
4. Alice sends the encrypted content and protected session key to Bob.

This approach combines the efficiency of symmetric encryption with the convenience of public-key cryptography.

Step 4: Bob decrypts the email

Bob uses his private key to recover the protected session key.

In simplified RSA notation:

$$M = C^{d} \bmod n$$

In a hybrid system, the operation recovers the session key rather than the complete email.

Bob then uses the recovered session key to decrypt the email content and verify its authentication tag.

Only someone who possesses the appropriate private key can recover the session key through the intended RSA decryption process.

Step 5: Ensure confidentiality and integrity

Encryption primarily provides confidentiality. It does not automatically prove who sent the message.

For sender authentication and integrity, Alice can digitally sign the email using her private signing key. Bob verifies the signature using Alice's authenticated public key.

For RSA signatures, modern implementations use a signature scheme such as RSA-PSS. Secure email standards may specify other approved signature constructions.

A digital signature helps Bob verify that the message was signed by the holder of Alice's private key and that the signed content has not been altered.

Important distinction:

- Encryption with Bob's public key protects confidentiality for Bob.
- Signing with Alice's private key supports sender authentication and integrity.
- A complete secure email design may need both.

4.2 Explain how key distribution and management maintain security.

Key distribution and management are critical because RSA security depends on the correct

association between identities and public keys and the protection of private keys.

A. Public-key distribution

Public keys may be distributed through:

- Digital certificates issued by trusted Certificate Authorities.
- An authenticated organizational directory.
- Securely maintained public-key repositories.
- Verified key fingerprints exchanged through a trusted channel.

A certificate binds an identity, such as Bob's email address, to a public key. The recipient verifies the certificate chain, validity period, identity information, and relevant revocation status according to the organization's trust policy.

If an attacker substitutes a fake public key for Bob's public key, Alice might encrypt confidential information for the attacker instead of Bob.

Therefore, public keys need not be secret, but their authenticity must be verified.

B. Private-key protection

Private keys must be protected against theft, unauthorized use, and disclosure.

Recommended controls include:

1. Strong operating-system and application access controls.
2. Encrypted key storage and secure backups.
3. Hardware Security Modules (HSMs) or other protected cryptographic storage where appropriate.
4. Multi-factor authentication for sensitive administrative operations.
5. Separation of duties and auditing for high-value organizational keys.

The private key should never be distributed in the same manner as the public key.

C. Key rotation and expiration

Keys should be replaced according to a defined cryptographic policy, including when they approach expiration or when their security may have been compromised.

Key rotation limits the consequences of long-term exposure and helps maintain compliance with organizational requirements.

However, rotating a key does not automatically protect previously encrypted messages if the corresponding private key remains compromised.

D. Revocation and compromise handling

If a private key is compromised, the organization should revoke the corresponding certificate or otherwise mark the key as untrusted, stop using it for new operations, investigate the incident, and replace it.

Certificate Revocation Lists (CRLs) and the Online Certificate Status Protocol (OCSP) are examples of mechanisms used to check certificate revocation status.

E. Backup and recovery

Private keys used for decrypting archived email require particular care. If such a key is permanently lost, previously encrypted messages may become unrecoverable.

An organization should maintain secure, access-controlled recovery procedures where required by its business and legal obligations.

Signing keys and encryption keys may need different backup and recovery policies because their purposes differ.

F. Use suitable key sizes and algorithms

The organization should follow current cryptographic standards for RSA key sizes, padding, certificates, and cryptographic libraries. For example, RSA keys of at least 2048 bits are commonly used for modern security requirements, while higher sizes may be selected according to the required protection period and applicable standards.

RSA should not be implemented using homemade cryptographic code or obsolete padding methods.

Conclusion: RSA can support confidential and authenticated email communication, but effective security depends on trusted public-key distribution, strong private-key protection, certificate validation, key rotation, revocation, and secure implementation.

5. Cryptography for IoT Devices

Scenario: A smart home system uses low-power IoT devices and must choose between RSA and Elliptic Curve Cryptography (ECC).

5.1 Analyze the constraints of IoT devices.

Internet of Things (IoT) devices include smart locks, temperature sensors, motion detectors, cameras, lighting controllers, and other connected devices.

Many of these devices have limited computational and energy resources. These constraints influence the choice of cryptographic algorithms.

A. Limited processing power

Low-cost microcontrollers may have relatively slow processors and limited hardware acceleration.

RSA public-key operations can be efficient for some operations, particularly encryption with a small public exponent, but RSA private-key operations and signatures can be comparatively expensive.

ECC generally achieves comparable security with much smaller keys and can offer efficient public-key operations on constrained devices, depending on the curve and implementation.

B. Limited memory

IoT devices often have restricted RAM and flash storage.

Large keys, certificates, cryptographic libraries, and temporary computational values can consume valuable memory.

ECC uses much smaller public keys and signatures than RSA at comparable security levels, helping reduce memory requirements and the amount of data that must be stored or transmitted.

C. Limited battery capacity

Many sensors operate on batteries and may need to remain active for months or years.

Expensive cryptographic operations increase energy consumption, especially when a device repeatedly establishes secure connections.

ECC can reduce communication overhead because of its smaller keys and messages, which may help reduce total energy consumption. The actual energy benefit depends on the processor, implementation, and frequency of communication.

D. Limited bandwidth

IoT devices may communicate over low-bandwidth wireless networks.

RSA public keys and signatures are generally larger than comparable ECC public keys and signatures. ECC can therefore reduce the amount of data transmitted during authentication and key establishment.

This is especially useful when a device sends short messages or communicates over a constrained wireless link.

E. Security requirements

Smart home systems may transmit sensitive information or control physical devices.

Examples include:

- Door-lock commands.
- Security-camera data.
- Motion-detection alerts.
- User authentication messages.
- Device configuration and firmware updates.

The cryptographic design must protect confidentiality, integrity, and device identity. It must also resist replay, impersonation, and other protocol-level attacks.

F. Scalability and maintenance

A smart home may contain dozens or hundreds of devices.

The system needs secure device enrollment, key provisioning, authentication, revocation, and updates. An algorithm alone cannot provide these capabilities; the surrounding key-management and communication protocols are equally important.

5.2 Recommend a suitable cryptographic approach and justify your answer.

The recommended approach is ECC for public-key cryptography, combined with a modern symmetric authenticated-encryption algorithm such as AES-GCM or an appropriately configured ChaCha20-Poly1305 implementation.

For example, an IoT device can use an authenticated key exchange based on elliptic-curve Diffie–Hellman (ECDH) to establish shared key material with a trusted controller or gateway.

The resulting key material is passed through a key derivation function and used to protect subsequent messages with symmetric authenticated encryption.

How ECC works in this context

ECC uses mathematical operations involving points on an elliptic curve over a finite field.

A private key is a secret scalar, while the public key is a corresponding elliptic-curve point.

In an elliptic-curve Diffie–Hellman exchange:

1. The IoT device generates a private scalar and calculates its public key.
2. The gateway or controller generates its own private and public key pair.
3. The parties exchange public keys.
4. Each party combines its own private key with the other party's public key to derive the same shared secret.
5. A key derivation function produces session keys.
6. The session keys protect communication using authenticated encryption.

The public keys must be authenticated to prevent man-in-the-middle attacks. This can be achieved through certificates, trusted pre-provisioned keys, or another suitable device-authentication mechanism.

Advantages of ECC for IoT

Feature	**RSA**	**ECC**
Key size for comparable security	Larger	Smaller
Bandwidth requirement	Generally higher	Generally lower
Memory requirements	Often higher	Often lower
Computational efficiency	Depends on operation; private-key operations can be costly	Often advantageous for constrained devices
Battery usage	Can be relatively high for repeated private-key operations	Often advantageous, but hardware and implementation matter

For an approximate comparison, a 256-bit ECC key provides security broadly comparable to RSA with a 3072-bit modulus under commonly used security estimates. Exact comparisons depend on the curve, implementation, and security assumptions.

Recommended implementation

A practical smart home design should use:

- ECDH: To establish shared secret material.
- A key derivation function: To derive cryptographic session keys.
- AES-GCM or ChaCha20-Poly1305: To encrypt and authenticate messages.
- Digital signatures or pre-provisioned trust: To authenticate devices and prevent unauthorized key exchange.
- Secure key storage: To protect long-term device credentials.
- Secure boot and signed firmware updates: To prevent unauthorized software from compromising the device.
- Replay protection: To prevent attackers from reusing captured commands or messages.

The system should use standardized, well-supported cryptographic protocols and implementations rather than designing its own protocol.

Why ECC is preferable to RSA for this scenario

ECC is generally preferable because it provides strong security with smaller keys and lower communication overhead. These properties are valuable when IoT devices have limited memory, bandwidth, processing capacity, and battery power.

RSA can still be appropriate in devices with sufficient resources or systems that require compatibility with existing RSA-based infrastructure. However, for a new smart home system consisting of low-power devices, ECC is usually the more practical public-key approach.

Conclusion: Choose ECC for key establishment and authentication, and use symmetric authenticated encryption for the actual data. This combination provides an effective balance between security, performance, and resource consumption.

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand